

EPAM SDET

Coverage Addendum + Follow-Up Question Bank

This addendum complements the previously generated 106-question master guide. It explicitly covers source questions that were too aggressively consolidated and provides a dedicated follow-up bank. The uploaded source contains repeated Round 1/2 lists plus additional framework/API/AI/Selenium material.

Important: "All possible follow-ups" is not literally finite. The follow-ups below are a broad practical set of likely interviewer probes for each topic.

Explicit source-gap / follow-up coverage

Q1. What is a Functional Interface?

Answer: A functional interface has exactly one abstract method; it can still have default and static methods. It is the target type for lambdas and method references.

```
@FunctionalInterface
interface Validator { boolean validate(String value); }
```

Likely follow-ups:

1. Which built-in functional interfaces do you know?
2. Can a functional interface have default methods?
3. How do you create a custom functional interface?
4. Why is @FunctionalInterface useful?

■ Know ■ Explain ■ Code ■ Follow-up

Q2. How do you average employee ages using a Map?

Answer: Stream the Map values, convert them to ints and call average(). Handle an empty map explicitly.

```
double avg = employees.values().stream()
    .mapToInt(Integer::intValue)
    .average().orElse(0.0);
```

Likely follow-ups:

1. What if the map contains null values?
2. How would you calculate average by department?
3. What is OptionalDouble?
4. Can you do it without streams?

■ Know ■ Explain ■ Code ■ Follow-up

Q3. Which Collection implementations have you used in Selenium automation?

Answer: Typical examples: List for ordered UI elements, Set for unique window handles, Map for test data/key-value data, Queue/Deque for work or retry queues. Choose based on access and concurrency requirements.

Likely follow-ups:

1. Why List instead of Set for product names?
2. Where have you used Map?
3. HashMap vs ConcurrentHashMap?
4. What collection would you use for window handles?

■ Know ■ Explain ■ Code ■ Follow-up

Q4. How do you read a .properties file?

Answer: Use java.util.Properties with an InputStream, close it with try-with-resources, and expose configuration through a dedicated config utility.

```
Properties p = new Properties();
try (InputStream in = new FileInputStream("config.properties")) {
    p.load(in);
}
String baseUrl = p.getProperty("baseUrl");
```

Likely follow-ups:

1. How do you override values from environment variables?
2. How do you handle missing keys?
3. Properties vs YAML?
4. How do you keep passwords out of the file?

■ Know ■ Explain ■ Code ■ Follow-up

Q5. Have you worked with Selenium Grid? Explain.

Answer: Selenium Grid distributes WebDriver sessions to remote browsers and supports parallel/cross-browser execution. Explain the Grid architecture you actually used and how capabilities were selected.

Likely follow-ups:

1. Grid 3 vs Grid 4?
2. How do you run 10 tests in parallel?
3. How do you isolate test data?
4. Grid vs BrowserStack/Sauce Labs?

■ Know ■ Explain ■ Code ■ Follow-up

Q6. What is Page Factory in Selenium?

Answer: PageFactory initializes WebElement fields annotated with @FindBy. It is often used with POM but is not POM itself.

Likely follow-ups:

1. What is @FindBy?
2. What are Page Factory disadvantages?
3. Does Page Factory solve stale elements?
4. Would you use it in a new Playwright framework?

■ Know ■ Explain ■ Code ■ Follow-up

Q7. What are the disadvantages of Page Factory?

Answer: It relies on proxy-based element initialization and can make locator lifecycle less explicit. Modern frameworks often prefer direct locators; Playwright's Locator API is a different model.

Likely follow-ups:

1. Page Factory vs plain By?
2. How does Playwright Locator differ?
3. How do you handle dynamic elements?
4. Can Page Factory improve maintainability?

■ Know ■ Explain ■ Code ■ Follow-up

Q8. How do you test broken links in parallel?

Answer: Collect HTTP links, partition work across workers, use thread-safe result storage and unique reporting. Respect server rate limits.

Likely follow-ups:

1. GET vs HEAD?
2. Which status codes count as broken?
3. How do you handle redirects?
4. How do you prevent too many concurrent requests?

■ Know ■ Explain ■ Code ■ Follow-up

Q9. Have you worked with API masking? Explain.

Answer: Mask sensitive values such as tokens, passwords and PII in logs/reports while retaining enough context for debugging. Apply redaction at logging/reporting boundaries.

Likely follow-ups:

1. How do you mask Authorization headers?
2. How do you protect PII in healthcare testing?
3. How do you verify masking works?
4. What should never be logged?

■ Know ■ Explain ■ Code ■ Follow-up

Q10. How do you read a file in REST Assured? Which class/import do you use?

Answer: Use `java.io.File` or `java.nio.file.Path` for request payloads; REST Assured can accept a `File` as the body. `JsonPath` is used for extracting JSON response values.

Likely follow-ups:

1. File vs Path?
2. How do you read a JSON field?
3. How do you handle missing files in Jenkins?
4. How do you parameterize file locations?

■ Know ■ Explain ■ Code ■ Follow-up

Q11. Write a REST Assured POST for an employee using Basic Authentication and a POJO.

Answer: Create an Employee POJO, authenticate with `auth().basic()`, set JSON content type, pass the POJO as the body and validate status and response fields.

```
given().auth().basic(user, password)
    .contentType(ContentType.JSON)
    .body(employee)
    .when().post("/employees")
    .then().statusCode(201);
```

Likely follow-ups:

1. Why use a POJO?
2. How is the POJO converted to JSON?
3. How do you validate the response body?
4. Where do credentials come from in CI?

■ Know ■ Explain ■ Code ■ Follow-up

Q12. What is a Hamcrest Matcher?

Answer: Hamcrest provides readable, composable matchers such as `equalTo`, `containsString`, `hasItem` and `greaterThan`. REST Assured commonly uses Hamcrest for assertions.

Likely follow-ups:

1. Hamcrest vs AssertJ?
2. Can you create a custom matcher?
3. Why are matchers useful in API validation?
4. Can REST Assured be used without Hamcrest?

■ Know ■ Explain ■ Code ■ Follow-up

Q13. How do you perform JSON Schema validation in REST Assured?

Answer: Validate the response structure and types against a JSON Schema, then add business assertions for important values.

Likely follow-ups:

1. Schema vs JsonPath?
2. Does schema validation verify business rules?
3. How do you version schemas?
4. What are required properties?

■ Know ■ Explain ■ Code ■ Follow-up

Q14. How do you manage test data?

Answer: Separate static data, generated data and environment configuration. Prefer API/database setup where appropriate, generate unique values for parallel execution, and clean up data.

Likely follow-ups:

1. JSON vs CSV vs database?
2. How do you isolate parallel tests?

3. How do you manage secrets?
4. How do you clean up created records?

■ Know ■ Explain ■ Code ■ Follow-up

Q15. How do you generate Allure reports?

Answer: Generate the report from allure-results and publish it through CI or serve it locally.

```
allure generate allure-results -o allure-report --clean
allure open allure-report
```

Likely follow-ups:

1. How do you attach screenshots?
2. How do you keep history?
3. How do you publish it in Jenkins?
4. What is the difference between results and report?

■ Know ■ Explain ■ Code ■ Follow-up

Q16. What reporting tools have you worked with?

Answer: The source mentions HTML, Extent and Allure. Explain what you used, what evidence each captures and how it integrates with CI.

Likely follow-ups:

1. Which report is easiest to publish in Jenkins?
2. How do you attach logs?
3. How do you trend failures?
4. How do you avoid oversized reports?

■ Know ■ Explain ■ Code ■ Follow-up

Q17. What is the difference between Maven profiles and plugins?

Answer: Plugins provide build/test functionality; profiles activate alternative configuration for environments or purposes such as regression. A profile can configure plugin behaviour.

Likely follow-ups:

1. Surefire vs Failsafe?
2. How do you activate a profile?
3. How do you pass system properties?
4. Where do you define plugins?

■ Know ■ Explain ■ Code ■ Follow-up

Q18. What is the difference between git fetch and git pull?

Answer: fetch downloads remote references without integrating them. pull fetches and then integrates, commonly by merge or rebase.

Likely follow-ups:

1. Can fetch change working files?
2. pull --rebase vs pull merge?
3. When would you fetch before merging?
4. How do you resolve conflicts?

■ Know ■ Explain ■ Code ■ Follow-up

Q19. What is the difference between git reset and git revert?

Answer: reset moves the branch pointer and may rewrite history; revert creates a new commit that undoes a previous commit and is safer for shared branches.

Likely follow-ups:

1. soft vs mixed vs hard reset?

2. Why avoid reset on main?
3. How do you undo a pushed commit?
4. How do you revert a merge commit?

■ Know ■ Explain ■ Code ■ Follow-up

Q20. Tell me about yourself.

Answer: Keep it to 60–90 seconds: 13+ years of quality engineering, automation leadership, web/API/mobile experience, current framework/CI ownership, migration work, domain experience, leadership and the role you are targeting.

Likely follow-ups:

1. Why EPAM?
2. Why are you looking for a change now?
3. What is your biggest automation achievement?
4. What makes you a lead rather than only a senior engineer?

■ Know ■ Explain ■ Code ■ Follow-up

Q21. If a developer rejects a bug, what is your approach?

Answer: Stay evidence-based: reproduce, provide steps, expected vs actual, requirement, environment and evidence. Clarify ambiguity with the product/requirement owner and document the final decision.

Likely follow-ups:

1. What if the developer still disagrees?
2. Who decides severity?
3. How do you avoid conflict?
4. Give a real example.

■ Know ■ Explain ■ Code ■ Follow-up

Q22. Describe the most challenging automation issue you solved.

Answer: Use STAR. A strong example is your Selenium-to-Playwright migration: explain the original problem, your ownership, migration/locator/wait/CI actions, and only use measurable results you can substantiate.

Likely follow-ups:

1. Why did you choose Playwright?
2. What was the hardest migration part?
3. How did you prove improvement?
4. What would you do differently?

■ Know ■ Explain ■ Code ■ Follow-up

Q23. Explain Test Plan.

Answer: A Test Plan defines scope, objectives, approach, environments, data, roles, schedule, risks, dependencies and entry/exit criteria for a project/release.

Likely follow-ups:

1. Who owns it?
2. How is it different from Test Strategy?
3. What are entry/exit criteria?
4. How do you handle changing scope?

■ Know ■ Explain ■ Code ■ Follow-up

Q24. Difference between Test Plan and Test Strategy.

Answer: Strategy is the higher-level quality/testing approach; the plan is more execution-oriented for a project/release. Terminology can vary by organisation.

Likely follow-ups:

1. Who defines strategy?
2. Can a project have its own plan?
3. Where do risks appear?
4. Give a real project example.

■ Know ■ Explain ■ Code ■ Follow-up

Q25. How do you calculate defect leakage?

Answer: Agree the denominator with stakeholders. A common formula is production/escaped defects divided by pre-production plus production defects, multiplied by 100.

Likely follow-ups:

1. What is defect escape?
2. How do you handle reopened defects?
3. How do you reduce leakage?
4. Why can formulas vary?

■ Know ■ Explain ■ Code ■ Follow-up

Q26. Explain defect leakage.

Answer: A defect leakage/escape is a defect not caught at the intended test stage and discovered later, including UAT or production.

Likely follow-ups:

1. How do you perform RCA?
2. What prevention actions follow?
3. Can leakage ever be zero?
4. How do you report leakage to management?

■ Know ■ Explain ■ Code ■ Follow-up

Q27. Explain Test Pyramid.

Answer: The pyramid favours many fast unit/component tests, fewer API/service tests and fewer expensive UI E2E tests. Automation selection should follow risk, feedback speed and maintainability.

Likely follow-ups:

1. Where do contract tests fit?
2. Why fewer UI tests?
3. How do you select UI candidates?
4. What if unit coverage is low?

■ Know ■ Explain ■ Code ■ Follow-up

Q28. How are you using Artificial Intelligence in day-to-day work?

Answer: Use AI for bounded tasks such as test generation/refactoring, debugging, documentation and log analysis. For your profile, mention Cursor and Claude with repository rules/skills rather than ad-hoc prompting. Review and execute generated code before committing.

Likely follow-ups:

1. How do you protect proprietary data?
2. How do you verify AI output?
3. What are hallucinations?
4. Where would you not use AI?

■ Know ■ Explain ■ Code ■ Follow-up

Q29. How do you handle dynamic elements?

Answer: Use stable attributes, accessible locators, parameterised relationships and condition-based waits. Avoid absolute XPath and fixed sleeps.

Likely follow-ups:

1. What if there are no stable attributes?
2. CSS vs XPath?
3. How do you handle changing text?
4. How do you coordinate test IDs with developers?

■ Know ■ Explain ■ Code ■ Follow-up

Q30. What is the difference between implicit, explicit and fluent waits?

Answer: Implicit is global element lookup waiting; explicit waits for a condition; FluentWait is an explicit wait with custom polling and ignored exceptions.

Likely follow-ups:

1. Why avoid mixing implicit and explicit waits?
2. What is polling?
3. What exceptions can be ignored?
4. How does Playwright auto-wait?

■ Know ■ Explain ■ Code ■ Follow-up

Q31. How do you handle authentication in Playwright?

Answer: Authenticate through UI or API, save storage state when appropriate, reuse it safely and isolate mutable test data/accounts for parallel execution.

Likely follow-ups:

1. How do you refresh auth?
2. Where do credentials live?
3. When should storage state not be shared?
4. How does this reduce execution time?

■ Know ■ Explain ■ Code ■ Follow-up

Q32. How do you use API and UI together in Playwright?

Answer: Use APIRequestContext for fast setup/teardown and backend validation, while UI tests validate user journeys. Keep data and authentication scoped to the test.

Likely follow-ups:

1. Why API setup instead of UI setup?
2. How do you share auth?
3. How do you correlate IDs?
4. How does this support the test pyramid?

■ Know ■ Explain ■ Code ■ Follow-up

Senior Interview Follow-Up Patterns

Definition probe: What is it? Why is it needed? What problem does it solve?

Implementation probe: Show me the code. Where exactly did you use it?

Trade-off probe: What are the disadvantages? When would you not use it?

Debugging probe: It works locally but fails in CI. How do you investigate?

Scale probe: What changes when hundreds of tests run in parallel?

Leadership probe: How did you make the decision and get team/stakeholder agreement?

Real-project probe: Give me an example from your current/previous framework.

Alternative probe: What alternatives did you consider and why did you choose this one?

Failure probe: What happens when this fails or becomes unavailable?

Measurement probe: How did you know the change actually improved quality/performance?

How to use this with the master PDF

Study the original master guide for the main answers and examples, then use this addendum as the completeness check. For each source question, practise the answer until you can handle at least the definition, implementation, trade-off and real-project follow-up without memorising wording.